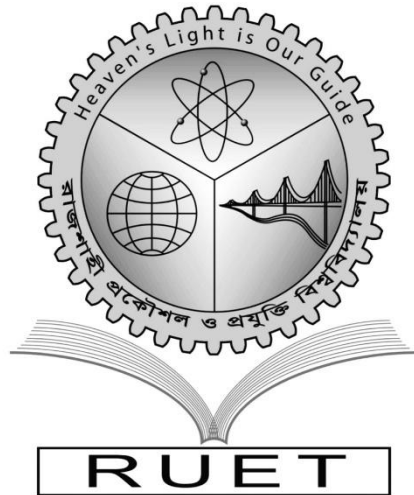


Heaven's light is our guide  
**Rajshahi University of Engineering & Technology**

**Department of Computer Science & Engineering**



**Lab Assignment**

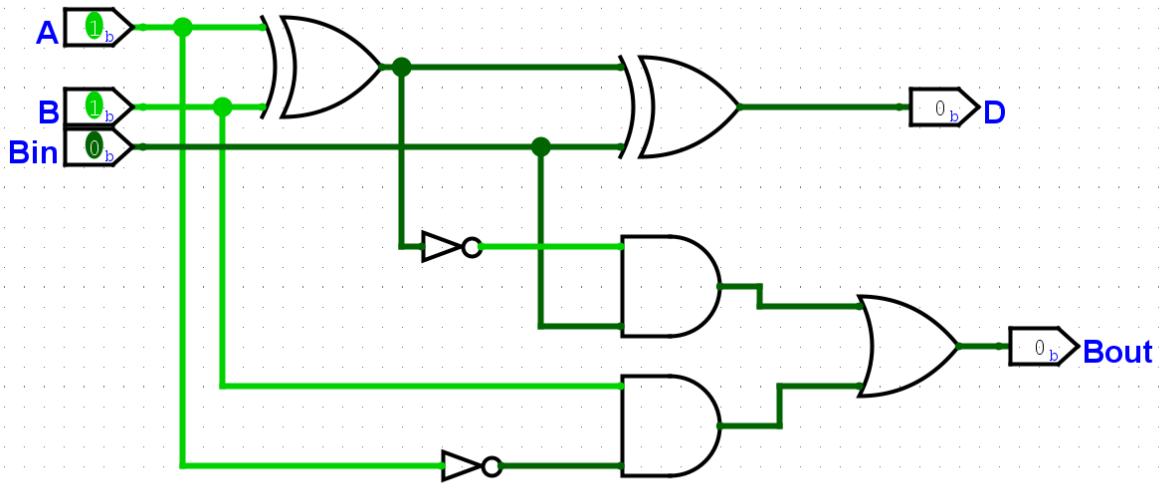
**Course Name :** Computer Architecture Sessional

**Course Code :** CSE 3108

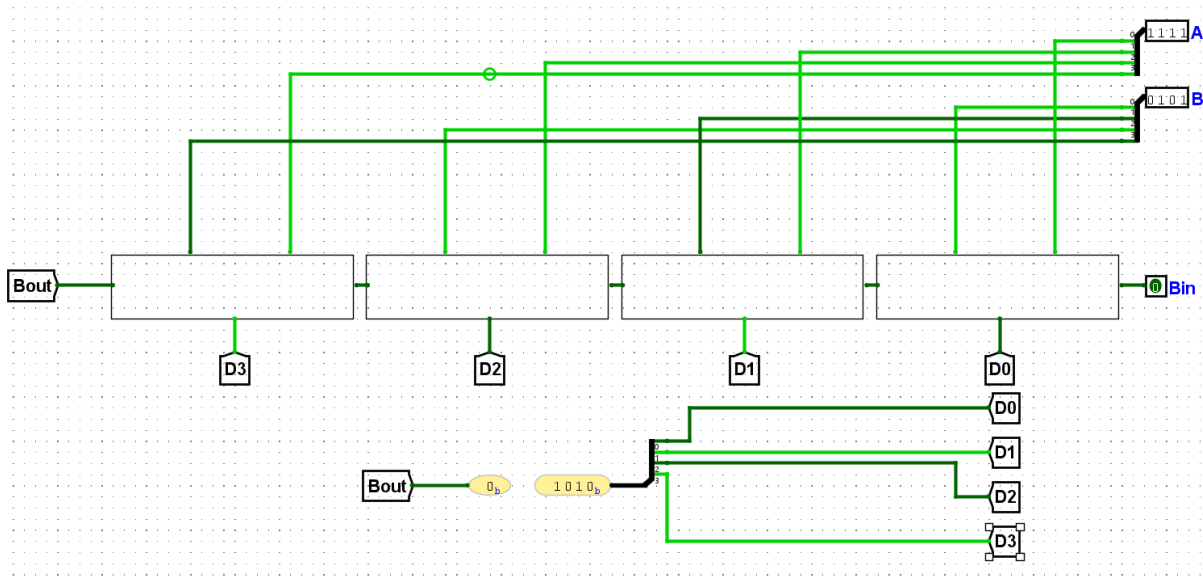
|                                                                                                         |                                                                                                                              |
|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>Submitted by :</b><br><br>Name : Md. Asad-Al-Adil<br>Roll no : 2103064<br>Section : B<br>Series : 21 | <b>Submitted to:</b><br><br><b>Utsha Das</b><br>Assistant Professor<br>Department of Computer Science &<br>Engineering, RUET |
|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|

## Set A

### 1. 4-Bit Full Parallel Subtractor using Logisim:



*1-bit Full Subtractor*



### 2. 4-bit Down Counter using HDL programming:

#### **register.v**

```
module register (
    input wire [3:0]d,
    input wire clk,
    input wire reset,
    output wire [3:0]q
```

```

);

reg [3:0]r_reg;
wire [3:0]r_next;

always @(posedge clk, posedge reset)
begin
    if (reset)
        r_reg<=4'b0000;
    else
        r_reg<=r_next;
end

assign r_next=d;
assign q=r_reg;

endmodule

```

#### **subone.v**

```

module subone (
    input wire [3:0]I,
    output wire [3:0]O
);

assign O=I-1;

endmodule

```

#### **counter.v**

```

module counter (
    input wire clk,
    input wire reset,
    output wire [3:0]q
);

wire [3:0]d_temp,q_temp;

```

```
register c1(d_temp,clk,reset,q_temp);
subone c2(q_temp,d_temp);

assign q=q_temp;

endmodule
```

### **cnt\_tb.v**

```
`timescale 1ns/1ps
module cnt_tb;
reg clk;
reg reset;
wire [3:0]q;

counter uut(clk,reset,q);

always begin
    clk=~clk;
    #10;
end

initial begin
    $dumpfile("test.vcd");
    $dumpvars(0,cnt_tb);

    clk<=0;
    reset<=1;
    #20;
    reset<=0;
    #20;#20;#20;#20;#20;#20;#20;#20;#20;#20;#20;#20;#20;
    #40;

    $finish;

end

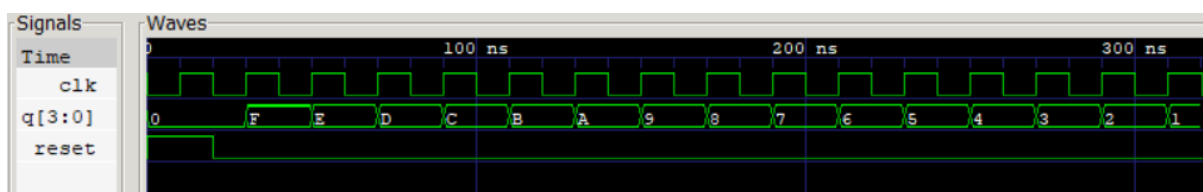
initial begin
    $monitor("reset=%b clk=%b q=%b",reset,clk,q);
```

```
end
```

```
endmodule
```

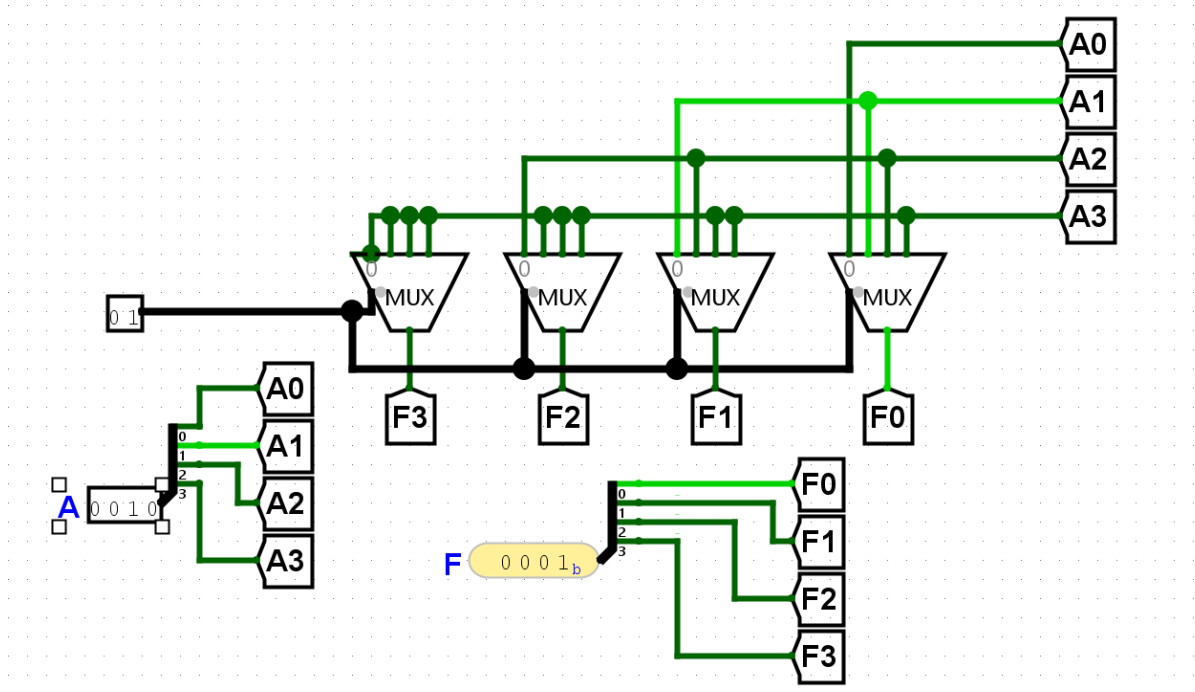
### Output:

```
VCD info: dumpfile test.vcd opened for output.  
reset=1 clk=0 q=0000  
reset=1 clk=1 q=0000  
reset=0 clk=0 q=0000  
reset=0 clk=1 q=1111  
reset=0 clk=0 q=1111  
reset=0 clk=1 q=1110  
reset=0 clk=0 q=1110  
reset=0 clk=1 q=1101  
reset=0 clk=0 q=1101  
reset=0 clk=1 q=1100  
reset=0 clk=0 q=1100  
reset=0 clk=1 q=1011  
reset=0 clk=0 q=1011  
reset=0 clk=1 q=1010  
reset=0 clk=0 q=1010  
reset=0 clk=1 q=1001  
reset=0 clk=0 q=1001  
reset=0 clk=1 q=1000  
reset=0 clk=0 q=1000  
reset=0 clk=1 q=0111  
reset=0 clk=0 q=0111  
reset=0 clk=1 q=0110  
reset=0 clk=0 q=0110  
reset=0 clk=1 q=0101  
reset=0 clk=0 q=0101  
reset=0 clk=1 q=0100  
reset=0 clk=0 q=0100  
reset=0 clk=1 q=0011  
reset=0 clk=0 q=0011  
reset=0 clk=1 q=0010  
reset=0 clk=0 q=0010  
reset=0 clk=1 q=0001  
cnt_tb.v:25: $finish called at 320000 (1ps)  
reset=0 clk=0 q=0001
```



## Set B

### 1. 4-bit Arithmetic Right Shift using Logisim:



### 2. 4-bit Full Subtractor using HDL programming

#### 1 bit sub.v

```
module sub1 (
    input wire A,
    input wire B,
    input wire Bin,
    output wire D,
    output wire Bout
);
```

```
assign D=A^B^Bin;
assign Bout=Bin&(~(A^B)) || (~A)&B;
```

#### 4-bit sub.v

```
module Full_sub (
    input wire [3:0]A,
```

```

input wire [3:0]B,
input wire Bin,
output wire [3:0]D,
output wire [3:0]Bout
);

sub1 s0(A[0],B[0],Bin,D[0],Bout[0]);
sub1 s1(A[1],B[1],Bout[0],D[1],Bout[1]);
sub1 s2(A[2],B[2],Bout[1],D[2],Bout[2]);
sub1 s3(A[3],B[3],Bout[2],D[3],Bout[3]);

endmodule

```

### **fs\_tb.v**

```

`timescale 1ns/1ps
module fs_tb;
reg [3:0]A;
reg [3:0]B;
reg Bin;
wire [3:0]Bout;
wire [3:0]D;

Full_sub uut(A,B,Bin,D,Bout);

initial begin
    $dumpfile("test.vcd");
    $dumpvars(0,fs_tb);
    A=4'b1000;
    B=4'b0011;
    Bin=0;
    #20;
    A=4'b1111;
    B=4'b0011;
    #20;
    A=4'b1001;
    B=4'b1001;
    #20;

```

```

A=4'b0111;
B=4'b1010;
#20;
#40;
end

initial begin

$monitor("A=%b, B=%b Bin=%b D=%b Bout=%b",A,B,Bin,D,Bout);
end

endmodule

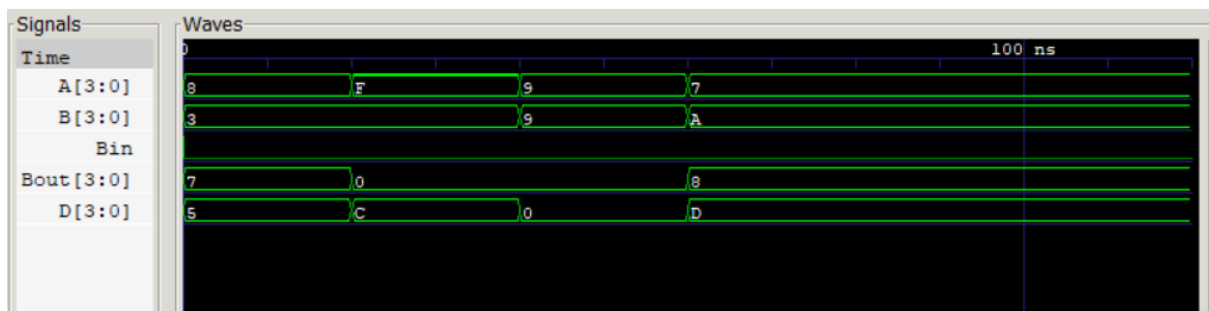
```

### Output:

```

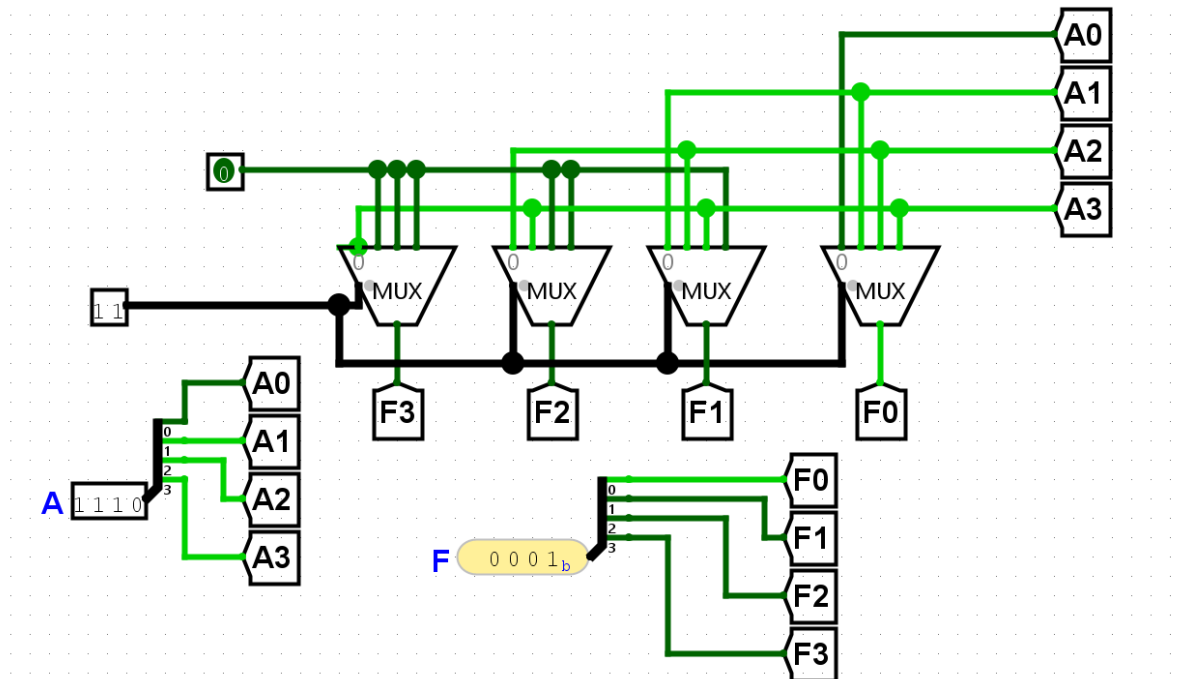
VCD info: dumpfile test.vcd opened for output.
A=1000, B=0011 Bin=0 D=0101 Bout=0111
A=1111, B=0011 Bin=0 D=1100 Bout=0000
A=1001, B=1001 Bin=0 D=0000 Bout=0000
A=0111, B=1010 Bin=0 D=1101 Bout=1000

```



## Set C

### 1. 4-bit Right Shift using Logisim



### 2. ARM Assembly Code for 10<sup>th</sup> Fibonacci Number:

```

;;; Directives
PRESERVE8
THUMB
; Vector Table Mapped to Address 0 at Reset
; Linker requires __Vectors to be exported
AREA RESET, DATA, READONLY
EXPORT __Vectors

```

```

__Vectors
    DCD 0x20001000
    ; stack pointer value when stack is empty
    DCD Reset_Handler ; reset vector
    ALIGN
    ; The program
    ; Linker requires Reset_Handler
    AREA MYCODE, CODE, READONLY
    ENTRY
    EXPORT Reset_Handler

```

Reset\_Handler

;;;;;;;;;;User Code Starts from the next  
line;;;;;;;;;;

MOV R0,#0

MOV R1,#1

MOV R2,#2

FOR

CMP R2,#10

BEQ DONE

ADD R3,R0,R1

MOV R0,R1

MOV R1,R3

ADD R2,R2,#1

B FOR

DONE

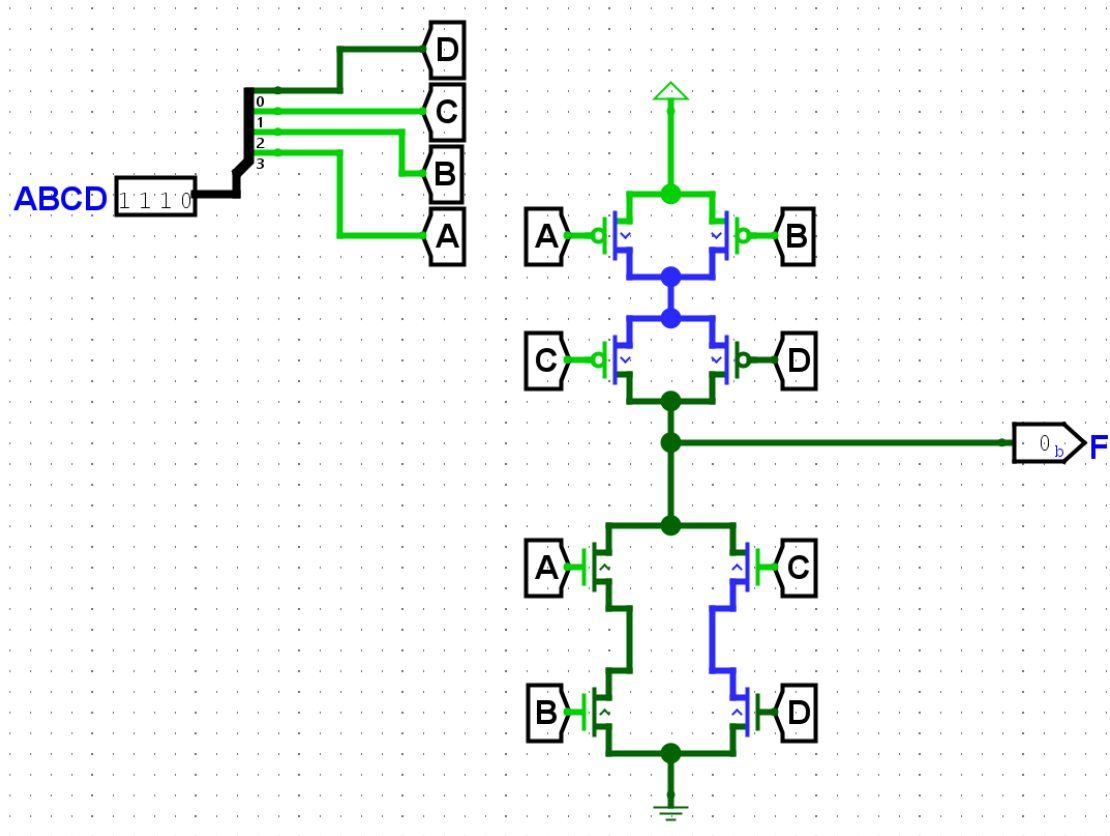
END

**Output:**

| Registers |            |
|-----------|------------|
| Register  | Value      |
| Core      |            |
| R0        | 0x00000015 |
| R1        | 0x00000022 |
| R2        | 0x0000000A |
| R3        | 0x00000022 |
| R4        | 0x00000000 |
| R5        | 0x00000000 |
| R6        | 0x00000000 |
| R7        | 0x00000000 |
| R8        | 0x00000000 |
| R9        | 0x00000000 |
| R10       | 0x00000000 |
| R11       | 0x00000000 |
| R12       | 0x00000000 |
| R13 (SP)  | 0x20001000 |
| R14 (LR)  | 0xFFFFFFFF |
| R15 (PC)  | 0x08010000 |
| xPSR      | 0x21000000 |

## Set D

1.  $F = (AB + CD)'$  using CMOS transistor in Logisim



2. 4 to 2 Priority Encoder using HDL programming

### circuit.v

```
module priority_encoder (  
    input wire [3:0]A,  
    output wire [1:0]Y  
);  
  
assign Y[0]=(~A[3])&(~A[2])&(A[1]) || A[3];  
assign Y[1]=(~A[3])&(A[2]) || A[3];  
  
endmodule
```

### tb.v

```
`timescale 1ns/1ps
```

```

module tb;
reg [3:0]A;
wire [1:0]Y;

priority_encoder uut(A,Y);

initial begin
    $dumpfile("test.vcd");
    $dumpvars(0,tb);

    A=4'b0111;
    #20;
    A=4'b1111;
    #20;
    A=4'b0011;
end

initial begin
    $monitor("A=%b Y=%b",A,Y);
end

endmodule

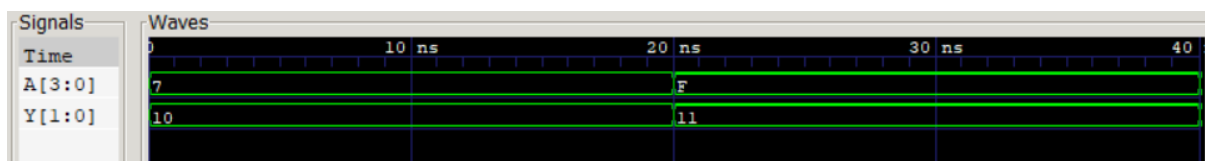
```

### Output:

```

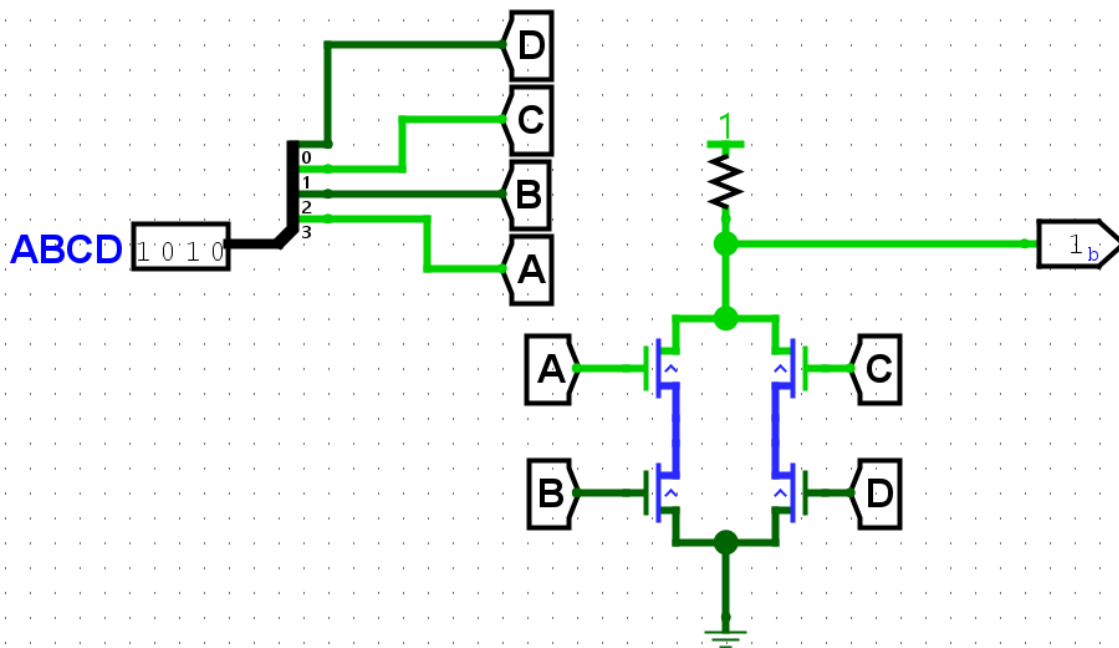
VCD info: dumpfile test.vcd opened for output.
A=0111 Y=10
A=1111 Y=11
A=0011 Y=01

```



## Set E

1.  $F = (AB + CD)'$  using NMOS transistor in Logisim



2. MOD-5 Counter using synchronous design methodology

### register.v

```
module register (  
    input wire [2:0]d,  
    input wire clk,  
    input wire reset,  
    output wire [2:0]q  
);  
  
reg [2:0]r_reg;  
wire [2:0]r_next;  
  
always @(posedge clk, posedge reset)  
begin  
    if (reset || r_reg==3'b100)  
        r_reg<=3'b000;  
    else  
        r_reg<=r_next;  
end
```

```
end

assign r_next=d;
assign q=r_reg;

endmodule
```

```
addone.v
module addone (
    input wire [2:0]I,
    output wire [2:0]O
);

assign O=I+1;

endmodule
```

```
counter.v
module counter (
    input wire clk,
    input wire reset,
    output wire [3:0]q
);

wire [2:0]d_temp,q_temp;

register c1(d_temp,clk,reset,q_temp);
subone c2(q_temp,d_temp);

assign q=q_temp;

endmodule
```

```
cnt_tb.v
`timescale 1ns/1ps
```

```
module cnt_tb;
reg clk;
reg reset;
wire [2:0]q;

counter uut(clk,reset,q);

always begin
    clk=~clk;
    #10;
end

initial begin
    $dumpfile("test.vcd");
    $dumpvars(0,cnt_tb);

    clk<=0;
    reset<=1;
    #20;
    reset<=0;
    #20;#20;#20;
    #40;

    $finish;

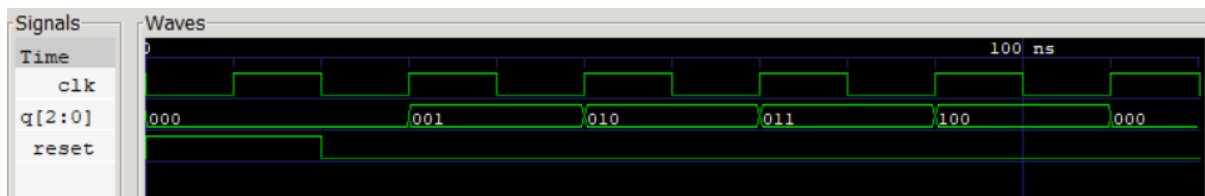
end

initial begin
    $monitor("reset=%b clk=%b q=%b",reset,clk,q);
end

endmodule
```

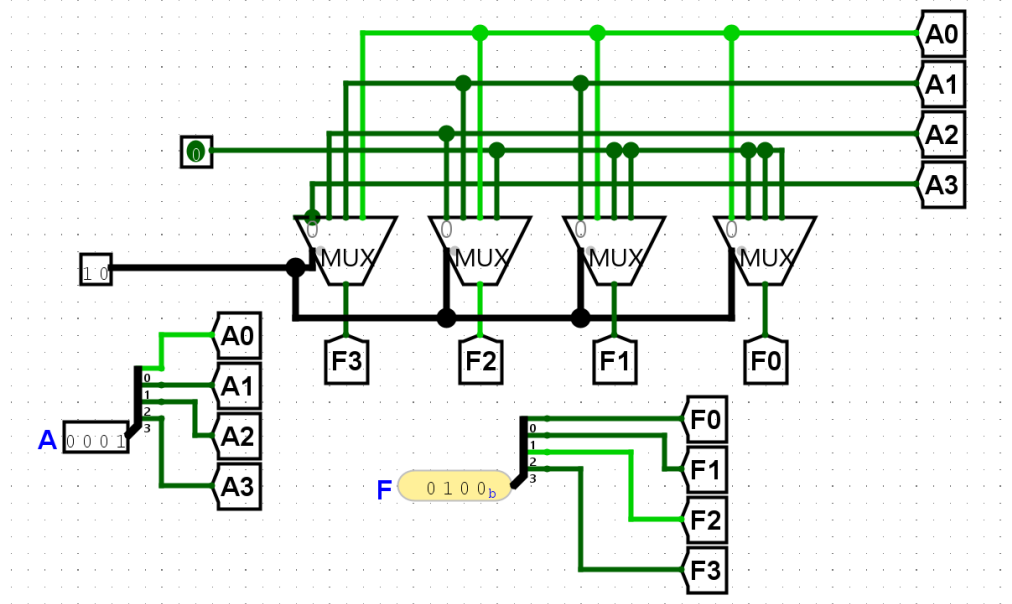
## Output:

```
VCD info: dumpfile test.vcd opened for output.  
reset=1 clk=0 q=000  
reset=1 clk=1 q=000  
reset=0 clk=0 q=000  
reset=0 clk=1 q=001  
reset=0 clk=0 q=001  
reset=0 clk=1 q=010  
reset=0 clk=0 q=010  
reset=0 clk=1 q=011  
reset=0 clk=0 q=011  
reset=0 clk=1 q=100  
reset=0 clk=0 q=100  
reset=0 clk=1 q=000  
cnt_tb.v:25: $finish called at 120000 (1ps)  
reset=0 clk=0 q=000
```



## Set F

### 1. 4-bit Left shift using Logisim



### 2. ARM Assembly code to calculate the summation of the 1<sup>st</sup> N elements of the following series, 1+3+5+7+.....(assume n=20)

```

;;; Directives
PRESERVE8
THUMB
; Vector Table Mapped to Address 0 at Reset
; Linker requires __Vectors to be exported
AREA RESET, DATA, READONLY
EXPORT __Vectors

__Vectors
    DCD 0x20001000
    ; stack pointer value when stack is empty
    DCD Reset_Handler ; reset vector
    ALIGN
    ; The program
    ; Linker requires Reset_Handler
    AREA MYCODE, CODE, READONLY
    ENTRY
    EXPORT Reset_Handler
    
```

```

Reset_Handler
;;;;;;;;;;User Code Starts from the next
line;;;;;;;;;;

    MOV R0,#1
    MOV R1,#1
    MOV R2,#1
FOR
    CMP R0,#20 ;N=20 assume
    BEQ DONE
    ADD R1,R1,#2
    ADD R2,R2,R1
    ADD R0,R0,#1
    B FOR
DONE
    END

```

## Output:

| Register    | Value      |
|-------------|------------|
| <b>Core</b> |            |
| R0          | 0x00000014 |
| R1          | 0x00000027 |
| R2          | 0x00000190 |
| R3          | 0x00000000 |
| R4          | 0x00000000 |
| R5          | 0x00000000 |
| R6          | 0x00000000 |
| R7          | 0x00000000 |
| R8          | 0x00000000 |
| R9          | 0x00000000 |
| R10         | 0x00000000 |
| R11         | 0x00000000 |
| R12         | 0x00000000 |
| R13 (SP)    | 0x20001000 |
| R14 (LR)    | 0xFFFFFFFF |
| R15 (PC)    | 0x08010000 |
| xPSR        | 0x21000000 |